

BSc SOFTWARE ENGINEERING
Second Year

Introduction to Cybersecurity Project

Course project report
compiled chapters 1–4

Team

Blessed

Favour

Amodi

Contents

1	STRIDE Threat Model for Secure Online Banking Architecture	1
1.1	System decomposition and trust boundaries	1
1.2	STRIDE threat analysis and mitigations	1
1.3	Defense-in-depth summary	4
1.4	Security gaps and continuous improvement	4
2	Secure Architecture Document	5
2.1	Network topology and firewall placement	5
2.2	TLS implementation plan	5
2.3	Key management strategy	6
3	ISO 27001 Control Mapping	8
3.1	Methodology	8
3.2	Control mapping and gap analysis	8
3.3	Strategic outcomes	8
4	Database Schema Design	11
4.1	Deployment platform	11
4.2	Design principles	11
4.3	Entity–relationship diagram	11
4.4	Relationship summary	12
4.5	Table definitions	12
4.5.1	users	12
4.5.2	accounts	14
4.5.3	transaction_states	14
4.5.4	transactions	15
4.5.5	transaction_entries	15
4.5.6	account_limits	16
4.5.7	audit_log	16
4.6	Indexing strategy	17
4.7	Security alignment summary	17

Chapter 1

STRIDE Threat Model for Secure Online Banking Architecture

This chapter presents a detailed STRIDE threat model for the secure online banking architecture. By decomposing the system into its core components—Frontend, Backend, and Data tiers—we identify potential vulnerabilities and map them to the security controls implemented in the project.

1.1 System decomposition and trust boundaries

The architecture is divided into distinct zones separated by *trust boundaries*, where data transitions between different levels of control.

Public zone

Includes Internet users (customers, tellers, admins) accessing the system via HTTPS/TLS 1.3.

DMZ (demilitarized zone)

Houses the frontend Python Flask server and the Web Application Firewall (WAF).

Private application zone

Contains the backend C# .NET Core Web API, which handles business logic and transaction processing.

Restricted data zone

The most secure tier, containing the Azure SQL Database and audit logs, protected by strict SQL firewalls.

Figure 1.1 gives an overview of the zoning model and trust boundaries discussed above.

1.2 STRIDE threat analysis and mitigations

Table 1.1 analyzes the banking system’s architecture through the STRIDE lens, identifying specific threats and the corresponding controls implemented to mitigate them.

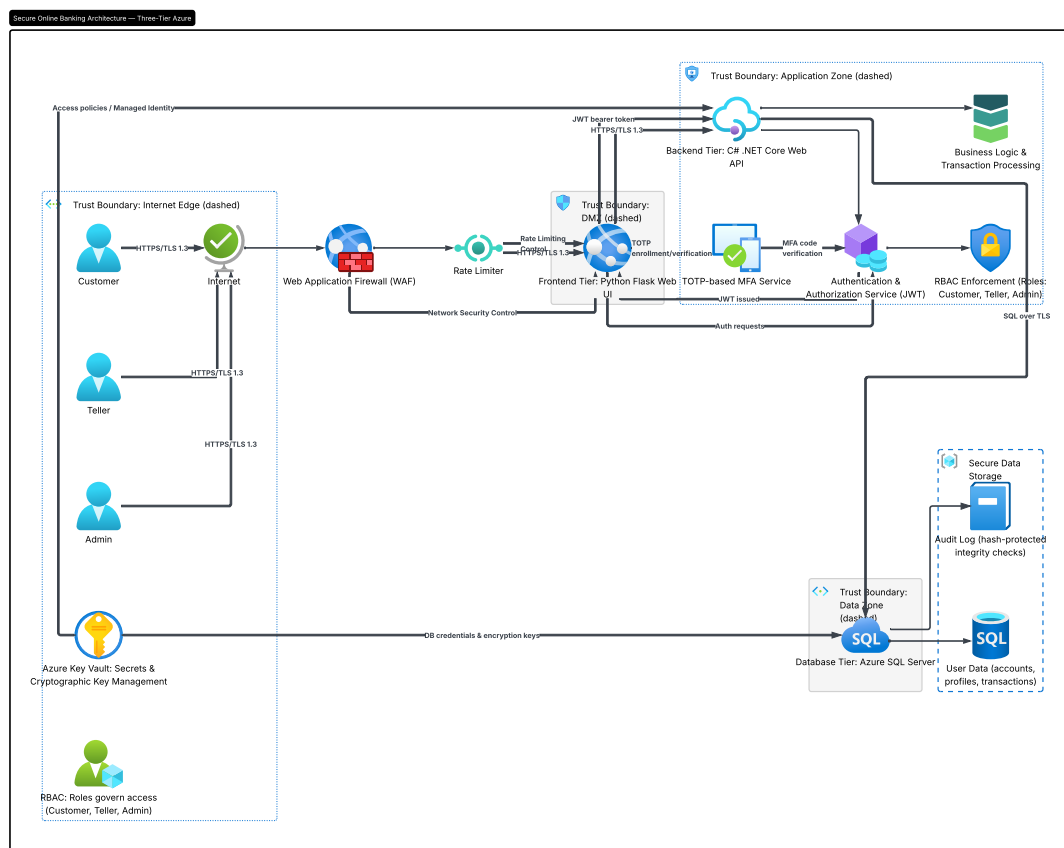


Figure 1.1: Trust boundaries and system decomposition (architecture overview).

Table 1.1: STRIDE mapping: threats and implemented mitigations

Threat category	Potential project risk	Implemented & compensating mitigations
Spoofing	Attackers stealing JWT tokens or bypassing MFA to impersonate a user.	JWT and TOTP MFA: JWT authentication and TOTP-based MFA; further mitigated by HttpOnly/Secure cookies and short token expiry.
Tampering	Unauthorized modification of transaction data or interception of data in transit.	TLS 1.3 and hashing: All communication encrypted via TLS 1.3. User data protected using bcrypt/Argon2 with unique random salts.
Repudiation	An admin or user performing an action and later denying it.	Audit logging: Separate audit servers with hash-protected integrity checks so logs are tamper-evident.
Information disclosure	Exposure of sensitive banking data via man-in-the-middle (MITM) attacks.	Azure Key Vault: Centralized management of secrets and encryption keys. HSTS used to prevent TLS downgrade attacks.
Denial of service	High-volume traffic or DDoS attacks overwhelming the banking API.	WAF and rate limiting: External firewalls provide DDoS mitigation; rate limiters at the edge restrict suspicious traffic spikes.
Elevation of privilege	A customer attempting to access admin-level transaction controls.	RBAC: Strict role-based access control (customer, teller, admin) enforced by the backend API and monitored via audit logs.

1.3 Defense-in-depth summary

To address the identified threats, the project employs a defense-in-depth strategy:

Network level: Segregation via internal and external firewalls and a DMZ prevents direct access to the database from the Internet.

Application level: The Web Application Firewall (WAF) protects against common exploits, complemented by robust input validation.

Identity level: RBAC and managed identities ensure that even if one component is compromised, lateral movement is limited by least-privilege policies.

1.4 Security gaps and continuous improvement

Although the current architecture is robust, the threat model highlights areas for strengthening—for example, implementing HSTS to prevent protocol downgrades and ensuring secure MFA recovery processes for users who lose access to their devices.

Chapter 2

Secure Architecture Document

This chapter details the defense-in-depth strategy for the secure online banking architecture. It explains how the three-tier Azure infrastructure is partitioned, how data is protected in transit, and how cryptographic secrets are managed throughout their lifecycle.

2.1 Network topology and firewall placement

The banking infrastructure uses a segmented network design to isolate sensitive data from public-facing components. Traffic is strictly filtered at each transition point so that a compromise in one zone does not grant automatic access to another.

Public zone (Internet edge)

The entry point for customers, tellers, and admins. Traffic is first filtered by an *external firewall* that provides DDoS mitigation and rate limiting to reduce the risk of service exhaustion.

Demilitarized zone (DMZ)

Houses the Web Application Firewall (WAF) and the frontend Python Flask server. The WAF inspects traffic for application-layer attacks.

Private application zone

Separated by an *internal firewall*, this zone contains the C# .NET Core Web API. This layer enforces least privilege: only validated requests from the frontend reach the business logic.

Restricted data zone

The most secure layer, containing the Azure SQL Database. Access is restricted by an Azure SQL firewall that allows only the specific IP addresses of the web API.

Figure 2.1 summarizes the segmented layout and trust boundaries for the banking infrastructure.

2.2 TLS implementation plan

To protect the confidentiality and integrity of banking data in transit, Transport Layer Security (TLS) 1.3 is enforced across communication channels.

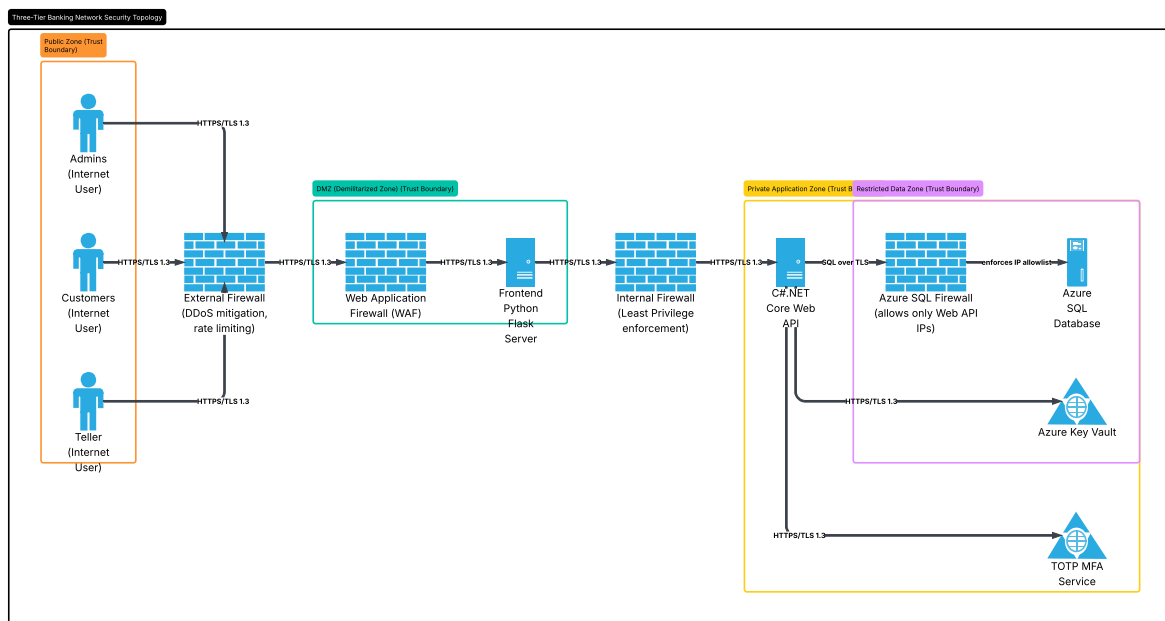


Figure 2.1: Network topology and firewall placement (reference diagram).

Implementation scope

User to frontend: All web traffic is encrypted via HTTPS/TLS 1.3 to protect login credentials and session tokens.

Service to service: Communication between the frontend UI and backend API uses TLS 1.3 to mitigate information disclosure.

Database connectivity: The system uses SQL over TLS for all database queries to reduce the risk of sniffing sensitive financial records.

Certificate and protocol hardening

The project treats downgrade attacks on older browsers as a risk. To mitigate this, HTTP Strict Transport Security (HSTS) headers are implemented so browsers connect only via HTTPS.

2.3 Key management strategy

Encryption depends on protecting cryptographic keys. This project centralizes cryptographic operations and secret storage in Azure Key Vault.

Secure storage and separation

Cryptographic keys and database credentials are stored in Key Vault, separate from application code and the database tier.

Access control and identity

- The system uses managed identities and role-based access control (RBAC) so that only authorized services (for example the backend API) can retrieve specific secrets.
- Least-privilege access policies are enforced to reduce the risk of broad credential theft.

Secret protection

Secure password hashing uses bcrypt/Argon2 with cryptographically secure random salts for every user.

Audit integrity

Access to keys is logged. To prevent a compromised administrator from destroying evidence, audit logs are stored on a separate server with hash-protected integrity checks to detect any alteration.

Chapter 3

ISO 27001 Control Mapping

This chapter maps the secure online banking platform to ISO/IEC 27001 and summarizes gaps and compensating controls.

3.1 Methodology

To align the banking platform with widely used security practice, we applied a three-step mapping process:

Inventory of existing controls: Identify defense-in-depth mechanisms already implemented in the Azure architecture.

Annex A alignment: Map those mechanisms to the 93 controls in ISO/IEC 27001:2022 Annex A.

Gap analysis and compensation: Identify cases where controls might be bypassed (for example zero-day exploits or token theft) and define compensating safeguards.

3.2 Control mapping and gap analysis

The complete mapping of technical controls to ISO/IEC 27001:2022 Annex A—including gap analysis and compensating controls—is provided in the PDF insert on the following pages. This insert is the authoritative record for this section.

3.3 Strategic outcomes

Mapping the architecture to these controls establishes a strong baseline for digital banking. Compensating measures such as HSTS and hash-protected logs reduce residual risk and keep the system resilient when primary controls face advanced threats.

Implemented Controls

- TLS (1.3)
- Firewalls
- Rate Limiter
- Key Vault (Separate server)
- Role Based Access Control
- JWT
- MFA (Multi Factor Authorisation)
- Hashing
- Audit logs(Separate Server)

<i>Existing Control</i>	<i>Annex A Mapping</i>	<i>Gap Identified</i>	<i>Compensating Control</i>	<i>Threats Mitigated Against</i>
TLS (1.3)	A.8.24 Use of Cryptography	Potential for downgrade attacks on older browsers.	Implement HSTS (HTTP Strict Transport Security) headers.+1	Information Disclosure, Tampering
Firewalls (WAF)	A.8.22 Segregation in Networks	Signature-based WAFs may miss "zero-day" exploits.	Input validation and output encoding at the application level.	Tampering, Denial of Service
Rate Limiter	A.8.26 Application Security Requirements +1	High-distributed attacks might bypass simple IP-based limits.	Account lockout policies and monitoring failed login attempts.+1	Denial of Service
Key Vault (Separate Server)	A.8.24 Use of Cryptography	Risk of "root" credential theft to the vault itself.	Least privilege access policies and managed identities for the API.+1	Information Disclosure, Elevation of Privilege
RBAC	A.5.15 Access Control	Manual role assignment can lead to human error.	Audit logging of all privilege changes and role assignments.+2	Elevation of Privilege, Spoofing
JWT	A.8.5 Secure Authentication	Token theft via Cross-Site Scripting (XSS).	HttpOnly and Secure flags for cookies; short expiration times.+1	Spoofing, Tampering
MFA (TOTP)	A.8.5 Secure Authentication	Users may lose access to their TOTP devices.	Secure Identity Recovery process with time-limited reset tokens.	Spoofing
Hashing (bcrypt/Argon2)	A.8.24 Use of Cryptography +1	Static salts can be vulnerable if	Use of cryptographically secure random salts for	Information Disclosure

Figure 3.1: ISO control mapping (full document, page 1 of 2).

		the salt generation is weak.	every unique user.	
Audit Logs (Separate Server)	A.8.15 Logging +2	Logs can be deleted by a compromised Admin account.	Hash-protected integrity checks to detect if log files have been altered.+1	Repudiation, Tampering

Figure 3.2: ISO control mapping (full document, page 2 of 2).

Chapter 4

Database Schema Design

This chapter presents the database schema for the secure online banking platform. The schema is implemented in **PostgreSQL** and enforces data integrity, financial accuracy, and auditability at the storage layer. Seven interrelated tables capture every aspect of the system—from user identity and account management to transaction processing and compliance logging.

4.1 Deployment platform

The original project design considered hosting the relational database on **Microsoft Azure** (for example Azure SQL Database). During setup we were **unable to create or activate an Azure account** (registration, verification, or subscription constraints). To continue the coursework without blocking on that provider, we deployed the same logical schema on **PostgreSQL** using **Neon’s free tier**. Neon provides managed PostgreSQL with TLS, a web SQL editor, and a connection string suitable for local development and demonstration. The DDL in `banking_schema.sql` is written for PostgreSQL and matches the tables created on Neon.

4.2 Design principles

The schema was designed around four guiding principles that align with the broader banking security design (identity controls, RBAC, encryption in transit, and auditability):

Referential integrity: Foreign-key constraints link every account to a user, every transaction to its accounts, and every audit entry to the actor who performed it. Orphaned records are structurally impossible.

Double-entry bookkeeping: Every financial movement produces two entries—a debit and a credit—ensuring that the ledger always balances and discrepancies are immediately detectable.

Least privilege and role separation: The `users` table stores an RBAC role (`customer`, `teller`, `manager`, `admin`) that the backend API should enforce at every endpoint.

Immutable audit trail: The `audit_log` table captures before/after snapshots of state changes, supporting non-repudiation and compliance review.

4.3 Entity–relationship diagram

Figure 4.1 shows the entity–relationship diagram exported from the schema design tool (dbdiagram.io), including tables and relationships as implemented in PostgreSQL.

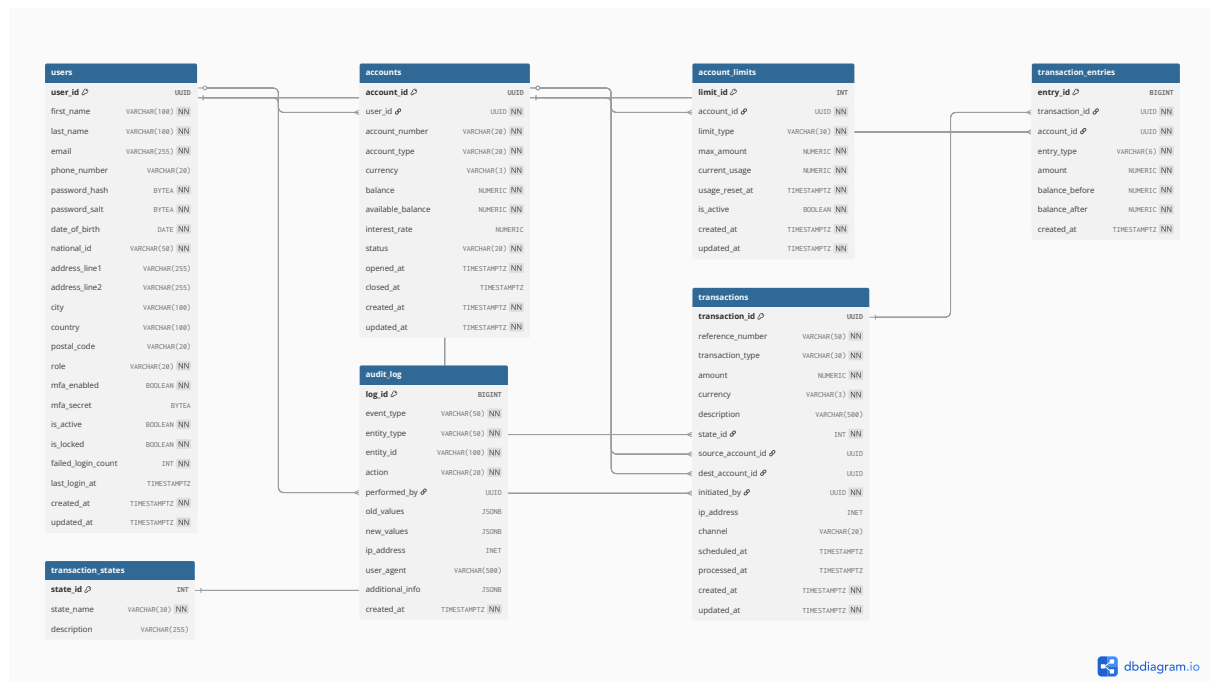


Figure 4.1: Entity–relationship diagram of the banking schema (PostgreSQL).

4.4 Relationship summary

Table 4.1 summarizes the relationships between the seven tables. Primary and foreign keys follow the naming used in `banking_schema.sql`.

Parent table	Child table	Relationship
users	accounts	One user owns zero or more accounts.
users	transactions	One user initiates zero or more transactions.
users	audit_log	One user generates zero or more log entries.
accounts	transactions	An account may be the source or destination of a transaction.
accounts	transaction_entries	An account has zero or more ledger entries.
accounts	account_limits	An account has zero or more spending limits.
transactions	transaction_entries	One transaction produces one or more ledger entries.
transaction_states	transactions	One state applies to zero or more transactions.

Table 4.1: Relationship summary for the banking schema.

4.5 Table definitions

Each subsection below describes one table: its purpose, columns, data types, constraints, and security rationale. The full SQL script is provided in the accompanying file `banking_schema.sql`.

4.5.1 users

The `users` table stores identity, authentication, and authorization data for every person who interacts with the platform.

Security notes. Passwords are stored as salted hashes using bcrypt or Argon2. The `is_locked` flag and `failed_login_count` enforce account lockout, mitigating brute-force attacks. UUIDs

Table 4.2: `users` table columns (PostgreSQL)

Column	Type	Constraint	Purpose
<code>user_id</code>	UUID	PK, default <code>gen_random_uuid()</code>	Non-sequential surrogate key.
<code>first_name</code>	VARCHAR(100)	NOT NULL	Legal first name.
<code>last_name</code>	VARCHAR(100)	NOT NULL	Legal last name.
<code>email</code>	VARCHAR(255)	UNIQUE, NOT NULL	Login identifier.
<code>phone_number</code>	VARCHAR(20)	NULL	Optional contact number.
<code>password_hash</code>	BYTEA	NOT NULL	Bcrypt/Argon2 hash (never plaintext).
<code>password_salt</code>	BYTEA	NOT NULL	Per-user cryptographic salt.
<code>date_of_birth</code>	DATE	NOT NULL	KYC age verification.
<code>national_id</code>	VARCHAR(50)	UNIQUE, NOT NULL	Government-issued ID for KYC.
<code>address fields</code>	VARCHAR	NULL	Line 1, Line 2, city, country, postal code.
<code>role</code>	VARCHAR(20)	CHECK	RBAC role: customer, teller, manager, admin.
<code>mfa_enabled</code>	BOOLEAN	DEFAULT FALSE	Whether MFA is active.
<code>mfa_secret</code>	BYTEA	NULL	Encrypted TOTP seed.
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Soft-delete flag.
<code>is_locked</code>	BOOLEAN	DEFAULT FALSE	Set after repeated failed logins.
<code>failed_login_count</code>	INT	DEFAULT 0	Counter for account-lockout policy.
<code>last_login_at</code>	TIMESTAMPTZ	NULL	Timestamp of most recent authentication.
<code>created_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Row creation time.
<code>updated_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Last modification time.

are used instead of sequential integers to reduce predictable identifier enumeration.

4.5.2 accounts

Each row in `accounts` represents a financial account owned by a user.

Table 4.3: `accounts` table columns (PostgreSQL)

Column	Type	Constraint	Purpose
<code>account_id</code>	UUID	PK, default <code>gen_random_uuid()</code>	Non-sequential surrogate key.
<code>user_id</code>	UUID	FK $\rightarrow$ <code>users</code>	Account owner.
<code>account_number</code>	VARCHAR(20)	UNIQUE	Customer-facing identifier.
<code>account_type</code>	VARCHAR(20)	CHECK	checking, savings, fixed_deposit, loan.
<code>currency</code>	VARCHAR(3)	DEFAULT USD	ISO 4217 currency code.
<code>balance</code>	NUMERIC(18,2)	DEFAULT 0	Ledger (book) balance.
<code>available_balance</code>	NUMERIC(18,2)	DEFAULT 0	Balance minus pending holds.
<code>interest_rate</code>	NUMERIC(5,4)	NULL	Annual rate for savings/loans.
<code>status</code>	VARCHAR(20)	CHECK	active, frozen, closed, dormant.
<code>opened_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Account opening time.
<code>closed_at</code>	TIMESTAMPTZ	NULL	Populated when account is closed.
<code>created_at</code> / <code>updated_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Row lifecycle timestamps.

Security notes. The `status` column allows the system to freeze an account immediately if fraud is detected, without deleting data. Splitting `balance` and `available_balance` prevents users from spending funds that are locked in pending transactions.

4.5.3 transaction_states

`transaction_states` is a reference table that defines the finite set of lifecycle states a transaction can occupy.

Table 4.4: `transaction_states` table columns and seed data

Column	Type	Constraint	Purpose
<code>state_id</code>	INT	PK, GENERATED ALWAYS AS IDENTITY	Surrogate key.
<code>state_name</code>	VARCHAR(30)	UNIQUE	Machine-readable state label.
<code>description</code>	VARCHAR(255)	NULL	Human-readable explanation.

The table is pre-seeded with seven states:

1. **pending** — transaction initiated, awaiting processing.
2. **authorized** — authorized but not yet settled.
3. **completed** — fully processed and settled.

4. **failed** — processing error occurred.
5. **reversed** — reversed after completion (e.g. chargeback).
6. **cancelled** — cancelled before settlement.
7. **on_hold** — flagged for manual review by compliance.

4.5.4 transactions

The **transactions** table records every financial operation in the system.

Table 4.5: **transactions** table columns (PostgreSQL)

Column	Type	Constraint	Purpose
transaction_id	UUID	PK, default <code>gen_random_uuid()</code>	Non-sequential surrogate key.
reference_number	VARCHAR(50)	UNIQUE	External reference for receipts.
transaction_type	VARCHAR(30)	CHECK	deposit, withdrawal, transfer, payment, fee, interest, refund.
amount	NUMERIC(18,2)	CHECK > 0	Transaction value.
currency	VARCHAR(3)	DEFAULT USD	ISO 4217 code.
description	VARCHAR(500)	NULL	Free-text memo.
state_id	INT	FK → transaction_states	Current lifecycle state.
source_account_id	UUID	FK → accounts	Debited account (NULL for deposits).
dest_account_id	UUID	FK → accounts	Credited account (NULL for withdrawals).
initiated_by	UUID	FK → users	Actor who started the transaction.
ip_address	INET	NULL	Client IP (native PostgreSQL type).
channel	VARCHAR(20)	CHECK	web, mobile, atm, branch, api.
scheduled_at	TIMESTAMPTZ	NULL	For future-dated transfers.
processed_at	TIMESTAMPTZ	NULL	When settlement completed.
created_at / updated_at	TIMESTAMPTZ	DEFAULT NOW()	Row lifecycle timestamps.

Security notes. Capturing **ip_address** and **channel** provides forensic context for fraud investigations. The **state_id** foreign key ensures that only valid, pre-defined states can be assigned.

4.5.5 transaction_entries

transaction_entries implements double-entry bookkeeping: every transaction generates at least two entries (a debit and a credit) so that the sum across all entries is always zero.

Security notes. Recording **balance_before** and **balance_after** creates an immutable ledger trail. If the running balance ever diverges from the sum of entries, the system can detect tampering or software bugs immediately.

Table 4.6: `transaction_entries` table columns (PostgreSQL)

Column	Type	Constraint	Purpose
<code>entry_id</code>	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	Surrogate key.
<code>transaction_id</code>	UUID	FK → <code>transactions</code>	Parent transaction.
<code>account_id</code>	UUID	FK → <code>accounts</code>	Affected account.
<code>entry_type</code>	VARCHAR(6)	CHECK	<code>debit</code> or <code>credit</code> .
<code>amount</code>	NUMERIC(18,2)	CHECK > 0	Entry value.
<code>balance_before</code>	NUMERIC(18,2)	NOT NULL	Account balance before this entry.
<code>balance_after</code>	NUMERIC(18,2)	NOT NULL	Account balance after this entry.
<code>created_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Immutable creation timestamp.

4.5.6 `account_limits`

`account_limits` enforces configurable spending and transfer caps per account, providing a technical control against fraud and unauthorized fund movement.

Table 4.7: `account_limits` table columns (PostgreSQL)

Column	Type	Constraint	Purpose
<code>limit_id</code>	INT	PK, GENERATED ALWAYS AS IDENTITY	Surrogate key.
<code>account_id</code>	UUID	FK → <code>accounts</code>	Account to which the limit applies.
<code>limit_type</code>	VARCHAR(30)	CHECK	<code>daily_withdrawal</code> , <code>daily_transfer</code> , <code>single_transaction</code> , <code>monthly_withdrawal</code> , <code>monthly_transfer</code> .
<code>max_amount</code>	NUMERIC(18,2)	CHECK > 0	Maximum permitted value.
<code>current_usage</code>	NUMERIC(18,2)	DEFAULT 0	Running total for the current period.
<code>usage_reset_at</code>	TIMESTAMPTZ	NOT NULL	When the counter resets.
<code>is_active</code>	BOOLEAN	DEFAULT TRUE	Enable or disable the limit.
<code>created_at</code> / <code>updated_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Row lifecycle timestamps.

A composite unique constraint on (`account_id`, `limit_type`) ensures that each account can have at most one active limit of each kind.

4.5.7 `audit_log`

The `audit_log` table provides an immutable compliance trail for security events and data changes.

Security notes. Storing `old_values` and `new_values` as JSONB enables indexed queries and point-in-time reconstruction. In production, append-only storage and integrity hashing of log

Table 4.8: `audit_log` table columns (PostgreSQL)

Column	Type	Constraint	Purpose
<code>log_id</code>	BIGINT	PK, GENERATED ALWAYS AS IDENTITY	Surrogate key.
<code>event_type</code>	VARCHAR(50)	NOT NULL	Category (e.g. <code>user.login</code>).
<code>entity_type</code>	VARCHAR(50)	NOT NULL	Table or resource affected.
<code>entity_id</code>	VARCHAR(100)	NOT NULL	Primary key of the affected row.
<code>action</code>	VARCHAR(20)	CHECK	create, read, update, delete, login, logout, failed_login.
<code>performed_by</code>	UUID	FK → <code>users</code>	Actor (NULL for system events).
<code>old_values</code>	JSONB	NULL	Snapshot before the change (queryable).
<code>new_values</code>	JSONB	NULL	Snapshot after the change.
<code>ip_address</code>	INET	NULL	Client IP for forensic correlation.
<code>user_agent</code>	VARCHAR(500)	NULL	Browser or API client identifier.
<code>additional_info</code>	JSONB	NULL	Free-form context (e.g. error payloads).
<code>created_at</code>	TIMESTAMPTZ	DEFAULT NOW()	Immutable event timestamp.

streams should complement this table design.

4.6 Indexing strategy

Performance-critical queries are supported by targeted indexes:

- **accounts:** indexed on `user_id` for fast lookup of all accounts belonging to a customer.
- **transactions:** indexed on `source_account_id`, `dest_account_id`, `state_id`, and `created_at` to accelerate statement generation, state-based filtering, and date-range queries.
- **transaction_entries:** indexed on `transaction_id` and `account_id` for efficient ledger reconstruction.
- **account_limits:** indexed on `account_id` so that limit checks during transaction processing are fast.
- **audit_log:** indexed on (`entity_type`, `entity_id`), `performed_by`, `created_at`, and `action` to support compliance queries and incident investigation.

4.7 Security alignment summary

Table 4.9 maps schema features to the threats they help mitigate.

Table 4.9: Schema features mapped to threats and controls

Schema feature	Threat addressed	Rationale
Salted password hashing	Spoofing (credential theft)	Passwords never stored in plaintext; slow hash resists offline guessing.
MFA columns	Spoofing (session hijacking)	Second factor reduces impact of stolen passwords.
RBAC role column	Elevation of privilege	Enforced in application layer against role escalation.
Account lockout fields	Spoofing (brute force)	Failed login counters and lock flags throttle guessing.
Double-entry bookkeeping	Tampering (balance manipulation)	Every movement has matching debit/credit entries.
Transaction state machine	Repudiation (state falsification)	States constrained by FK to reference data.
Audit log with JSON snapshots	Repudiation (action denial)	Actor, action, and before/after values recorded.
IP and user-agent logging	Forensics / incident response	Correlates sessions with security events.
Account limits	Fraud / fund drain	Caps velocity of withdrawals and transfers.
UUID primary keys	Enumeration	Non-sequential identifiers harder to guess than integers.